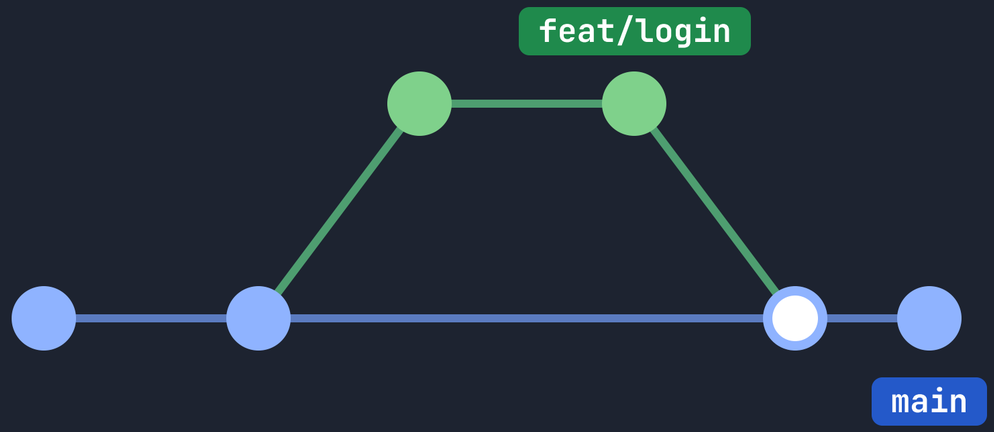
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git 브랜치, 눈으로 보기

따로 작업하고, 합치고,
충돌을 푸는 법



오늘의 순서

1
브랜치란?
왜 쓰는지
이름표와 HEAD

3–4쪽

2
만들고 합치기
checkout -b · merge
이름 규칙

5–13쪽

3
갈라진 길 합치기
merge 커밋

14–17쪽

4
충돌 풀기
충돌 표시 읽기
해결하고 커밋

18–21쪽

5
GitHub에서 함께
push · Pull Request
GitHub Flow

22–26쪽

6
막힐 때
자주 나는 에러

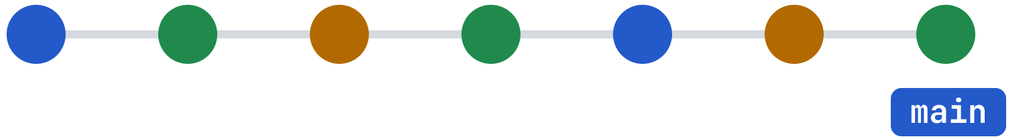
27–29쪽

마지막 30쪽은 한 장 요약이에요.

브랜치란?

왜 브랜치를 쓸까요?

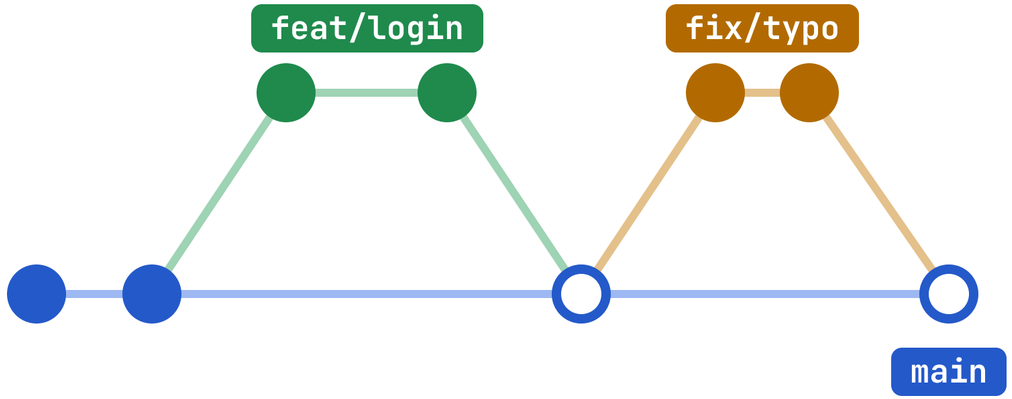
⚠ 모두 main에 바로 커밋



반쯤 만든 기능이 main에 섞여요

누구 작업이 문제인지 찾기 어려워요

✓ 각자 브랜치에서 작업



다 만든 다음에 main으로 합쳐요

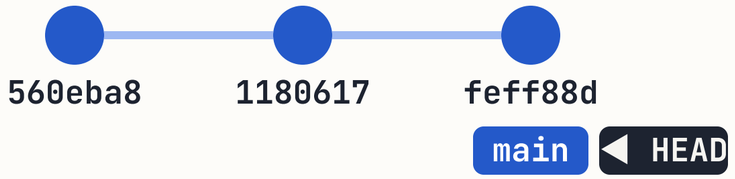
main은 늘 잘 돌아가는 상태로 남아요

브랜치 = 본 줄기(main)는 그대로 두고, 옆으로 갈래를 내서 작업하는 방법

실습 1 · 지금 모습 보기

git log --oneline --decorate

커밋 그래프



작업 폴더

hello.md

```
hello
world
```

README.md

```
# hello-git
```

```
$ git log --oneline --decorate
feff88d (HEAD → main) docs: README 추가
1180617 feat: 인사말 추가
560eba8 feat: 인사 파일 추가
```

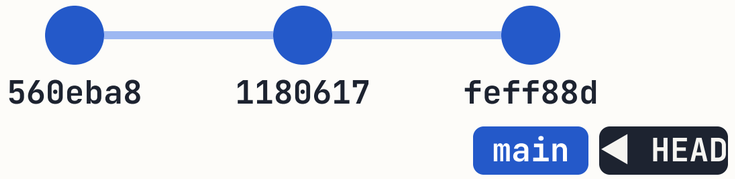
main = 마지막 커밋에 붙은 이름표

HEAD = 지금 내가 있는 브랜치

실습 2 · 브랜치 목록

git branch

커밋 그래프



작업 폴더

```
hello.md
hello
world
```

```
README.md
# hello-git
```

```
$ git branch
* main
```

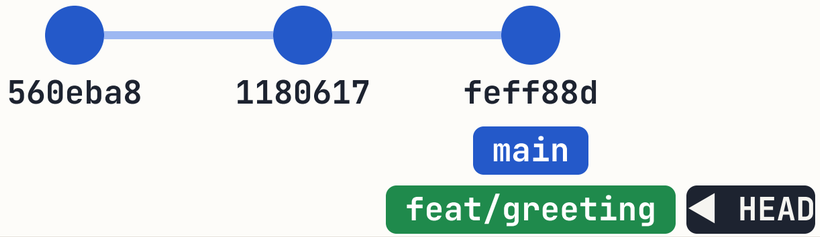
브랜치 목록 보기

* 표시가 지금 있는 브랜치

실습 3 · 브랜치 만들고 옮겨 가기

git checkout -b feat/greeting

커밋 그래프



작업 폴더

```
hello.md
hello
world
```

```
README.md
# hello-git
```

```
$ git checkout -b feat/greeting
Switched to a new branch 'feat/greeting'
$ git branch
* feat/greeting
  main
```

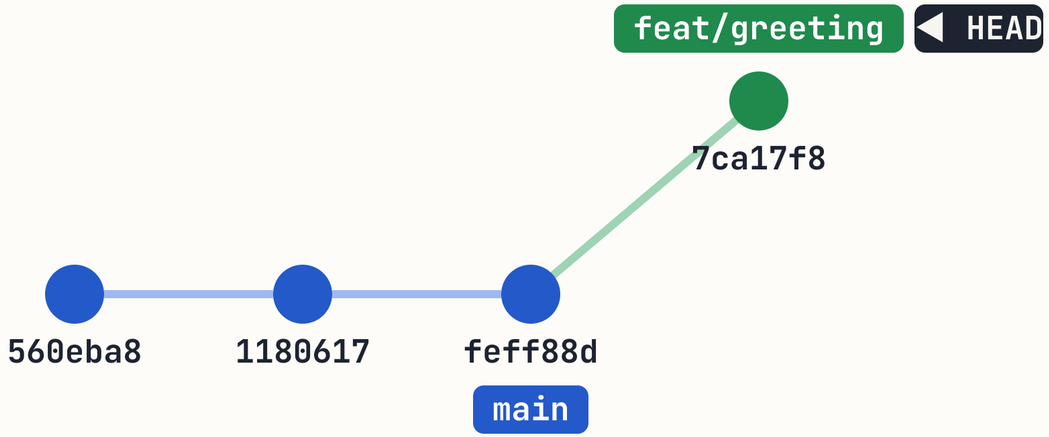
이름표가 하나 더 생겼어요

커밋도 파일도 그대로예요

실습 4 · 브랜치에서 커밋

git commit -m "feat: 반가움 인사 추가"

커밋 그래프



작업 폴더

hello.md
hello
world
nice to meet you

README.md

```
# hello.md에 한 줄 추가
$ git add hello.md
$ git commit -m "feat: 반가움 인사 추가"
[feat/greeting 7ca17f8] feat: 반가움 인사 추가
 1 file changed, 1 insertion(+)
```

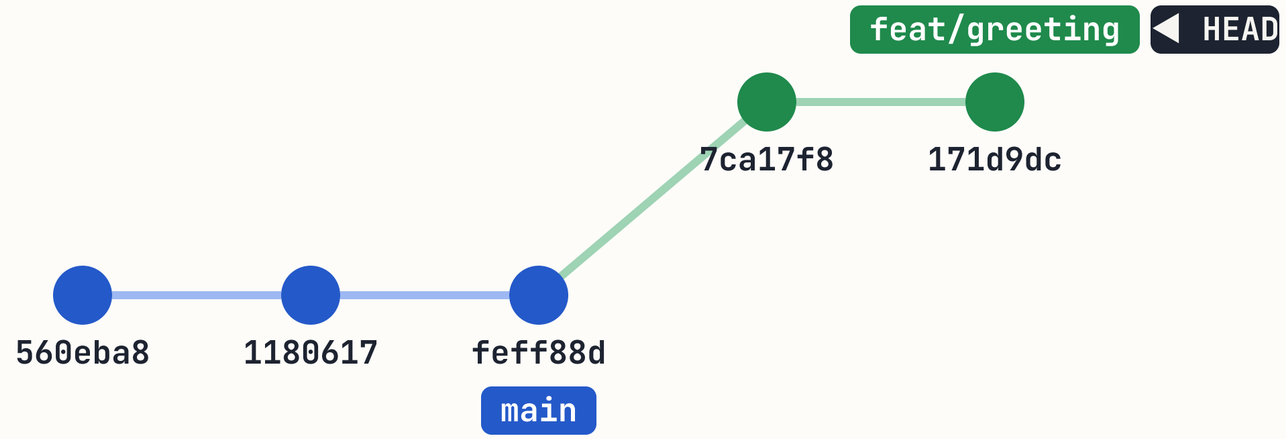
feat/greeting만 앞으로 가요

main은 제자리

실습 5 · 한 번 더 커밋

git commit -m "feat: 끝인사 파일 추가"

커밋 그래프



작업 폴더

bye.md
see you

hello.md

README.md

```
# 새 파일 bye.md
$ git add bye.md
$ git commit -m "feat: 끝인사 파일 추가"
[feat/greeting 171d9dc] feat: 끝인사 파일 추가
 1 file changed, 1 insertion(+)
 create mode 100644 bye.md
```

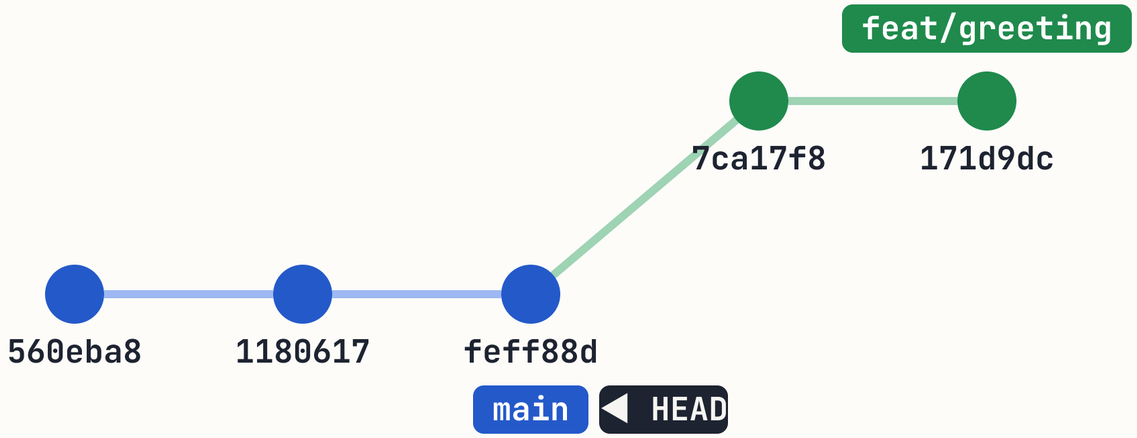
브랜치 위에 커밋이 쌓여요

main에는 아직 아무 영향 없음

실습 6 · main으로 돌아가기

git checkout main

커밋 그래프



작업 폴더

```
hello.md
hello
world
```

```
README.md
```

~~bye.md~~

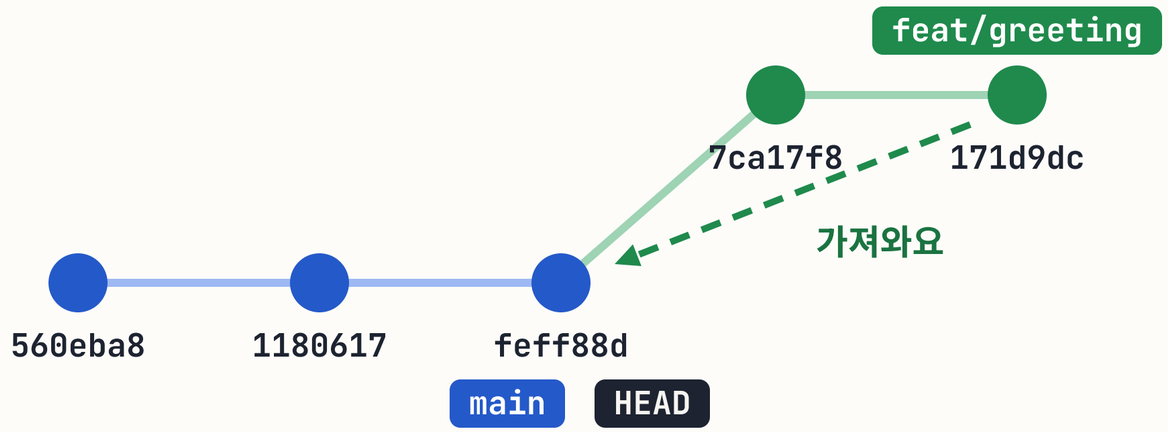
```
$ git checkout main
Switched to branch 'main'
$ ls
README.md hello.md
```

파일도 main 시점으로 바뀌어요

bye.md는 사라진 게 아니라 브랜치에 있어요

만들고 합치기 · merge

merge는 받는 쪽에서 해요



받는 쪽

먼저 checkout으로 가 있는 브랜치

가져올 쪽

git merge 뒤에 적는 이름

1 받을 브랜치로 이동

```
git checkout main
```

2 가져올 브랜치 이름을 적어요

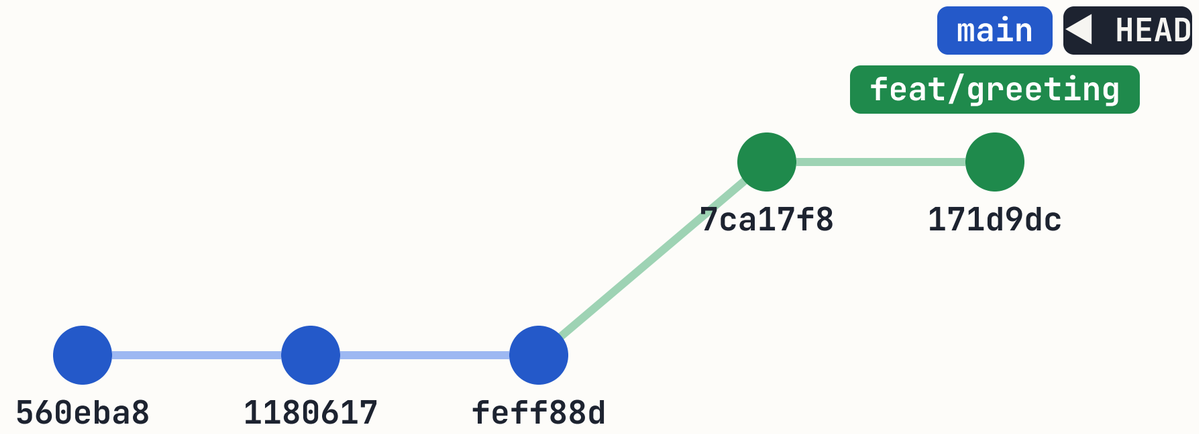
```
git merge feat/greeting
```

02에서 본 git pull origin main도 받아 온 커밋을 이 방식으로 merge해요.

실습 7 · 브랜치 합치기

git merge feat/greeting

커밋 그래프



작업 폴더

hello.md

```
hello
world
nice to meet you
```

bye.md

```
see you
```

README.md

```
$ git merge feat/greeting
Updating feff88d..171d9dc
Fast-forward
 bye.md | 1 +
 hello.md | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 bye.md
```

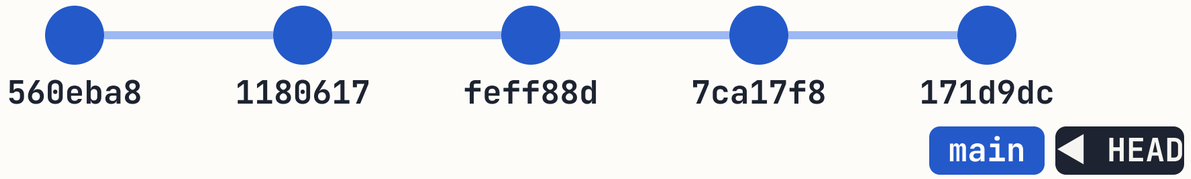
Fast-forward: 이름표만 앞으로

main 쪽에 새 커밋이 없어서
합칠 게 없었어요

실습 8 · 다 쓴 브랜치 지우기

git branch -d feat/greeting

커밋 그래프



작업 폴더

- hello.md
- README.md
- bye.md

```
$ git branch -d feat/greeting
Deleted branch feat/greeting (was 171d9dc).
$ git branch
* main
```

이름표만 떼요

커밋은 main에 한 줄로 남아요

만들고 합치기 · 이름 짓기

브랜치 이름은 타입/하는 일

feat/login-button

타입 · 슬래시 · 하는 일(소문자, 하이픈)

feat/	fix/	docs/	refactor/	chore/
새 기능	버그 수정	문서	구조 개선	설정, 패키지
feat/login-button	fix/signup-error	docs/readme-install	refactor/user-api	chore/eslint-setup

이렇게

- feat/login-button
- fix/12-signup-error (이슈 12번)

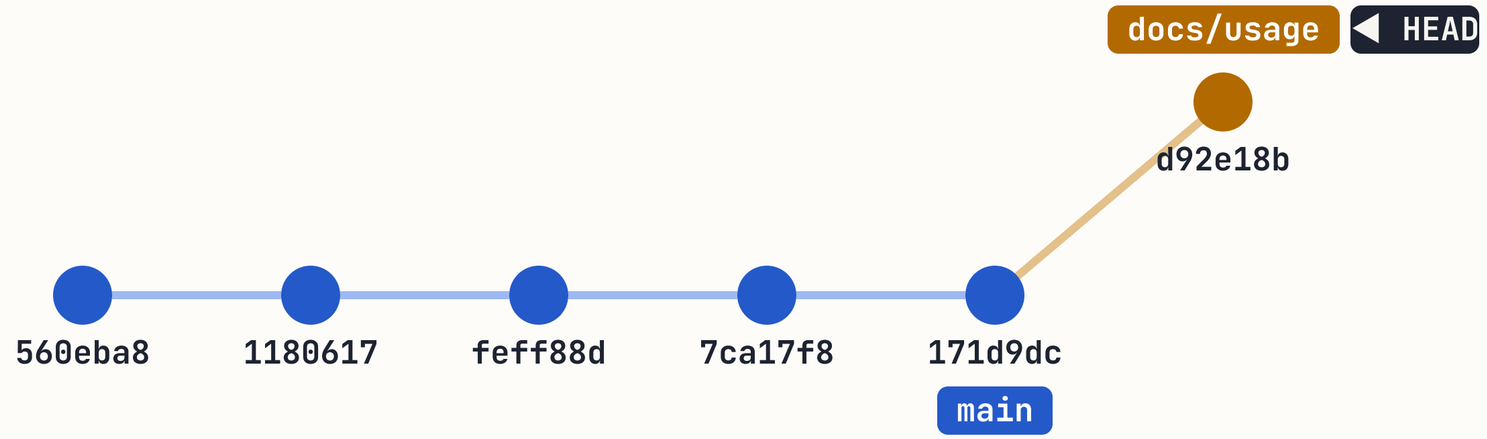
이건 피해요

- test, 내작업, final2 (무슨 일인지 모름)
- Feat/Login Button (대문자, 띄어쓰기)

실습 9 · 새 브랜치에서 커밋

git checkout -b docs/usage

커밋 그래프



작업 폴더

README.md
hello-git

인사 파일 모음

hello.md

bye.md

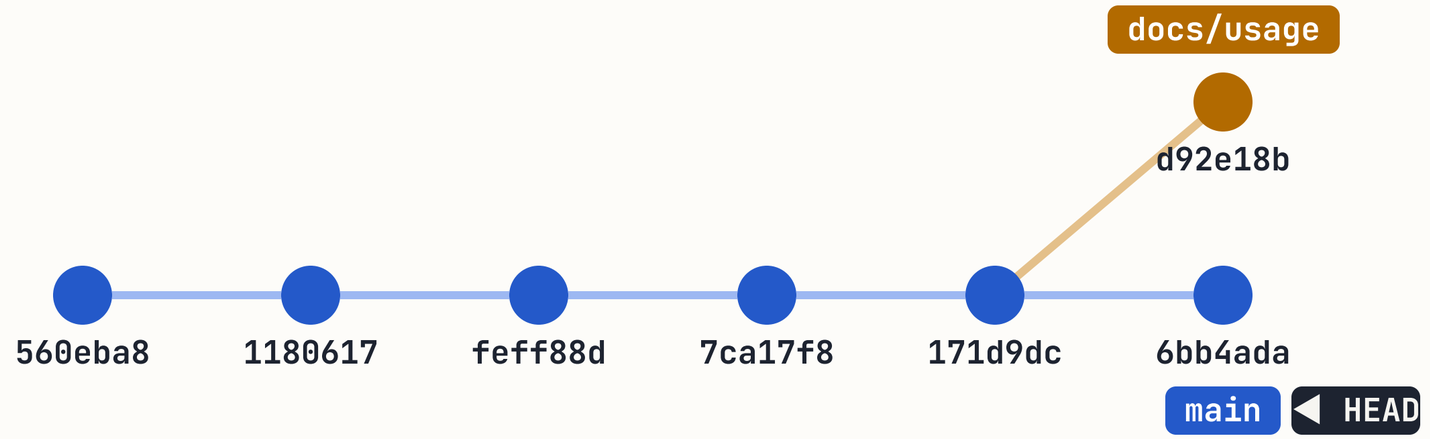
```
$ git checkout -b docs/usage
Switched to a new branch 'docs/usage'
# README.md에 소개 추가
$ git add README.md
$ git commit -m "docs: README에 소개 추가"
[docs/usage d92e18b] docs: README에 소개 추가
 1 file changed, 2 insertions(+)
```

이번엔 docs/usage에서 README 작업

실습 10 · 그 사이 main에도 커밋

git commit -m "feat: 끝인사 한 줄 추가"

커밋 그래프



작업 폴더

bye.md
```
see you
bye
```

hello.md

README.md

```
$ git checkout main
Switched to branch 'main'
# bye.md에 한 줄 추가
$ git add bye.md
$ git commit -m "feat: 끝인사 한 줄 추가"
[main 6bb4ada] feat: 끝인사 한 줄 추가
 1 file changed, 1 insertion(+)
```

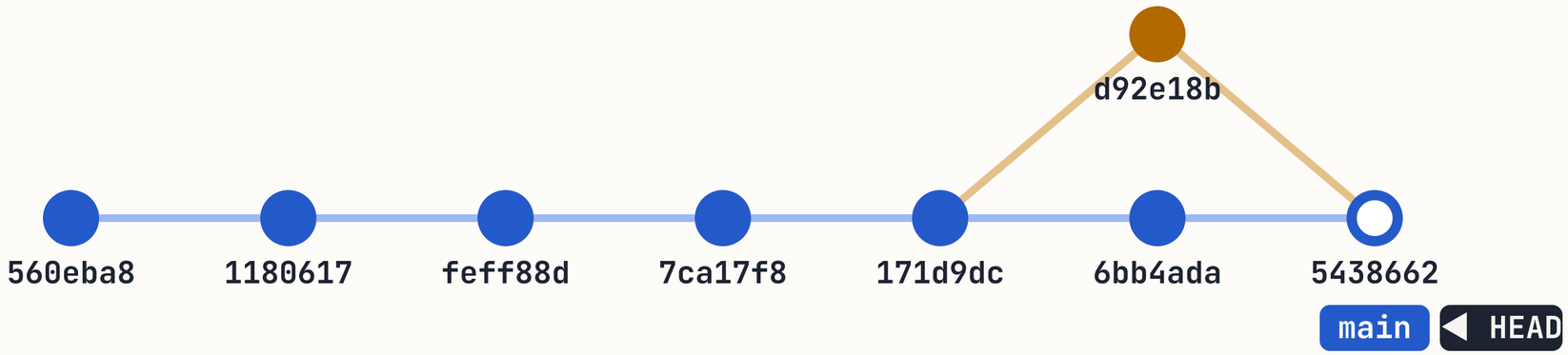
길이 두 갈래로

main에도, docs/usage에도
새 커밋이 있어요

실습 11 · 갈라진 길 합치기

git merge docs/usage

커밋 그래프



작업 폴더

README.md
```
# hello-git

인사 파일 모음
```

bye.md
```
see you
bye
```

hello.md

```
$ git merge docs/usage
# 메시지 편집기가 열리면 그대로 저장하고 닫기
Merge made by the 'ort' strategy.
 README.md | 2 ++
 1 file changed, 2 insertions(+)
$ git branch -d docs/usage
Deleted branch docs/usage (was d92e18b).
```

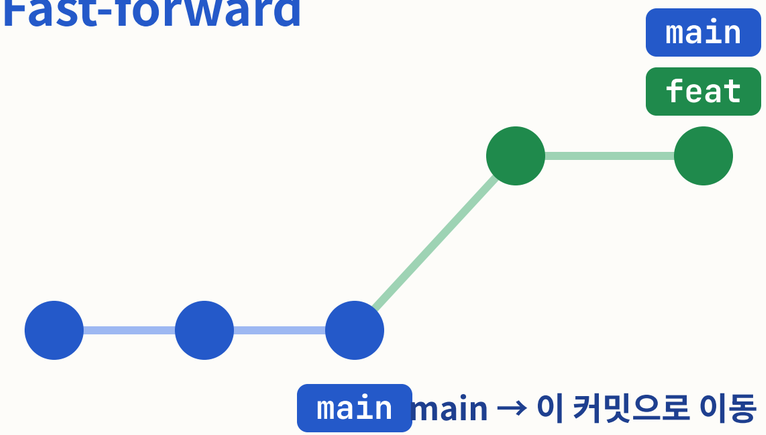
두 줄기를 잇는 merge 커밋

편집기가 vim이면 :wq 로 저장하고 닫아요

개념 정리

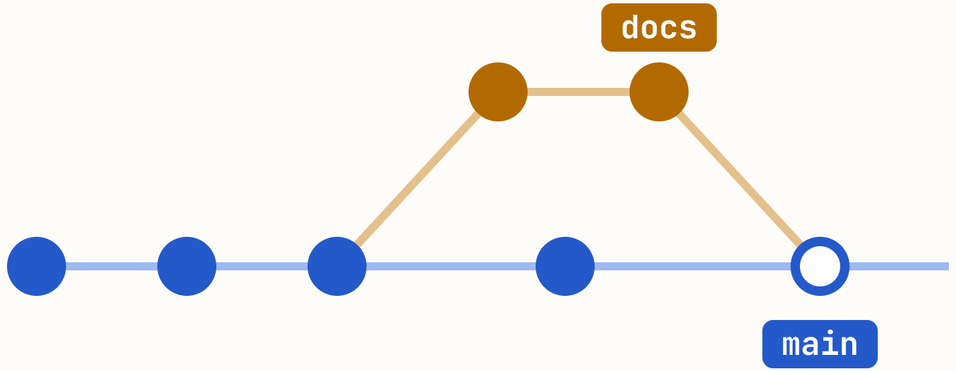
merge는 두 가지 모습으로 끝나요

Fast-forward



main에 새 커밋이 없을 때
이름표만 앞으로 당겨요

merge 커밋



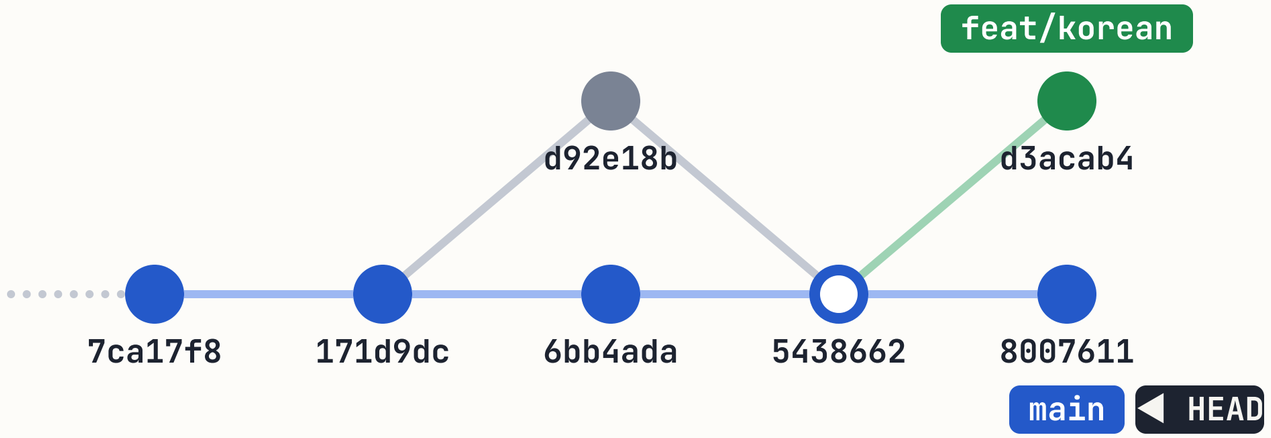
양쪽에 새 커밋이 있을 때
둘을 잇는 커밋을 새로 만들어요

명령은 똑같이 git merge 브랜치이름. 어느 쪽이 될지는 Git이 정해요.

실습 12 · 같은 줄을 서로 다르게

hello.md 첫 줄: hi vs 안녕

커밋 그래프



첫 줄이 달라요

main의 hello.md

```
hi
world
nice to meet you
```

feat/korean의 hello.md

```
안녕
world
nice to meet you
```

```
$ git checkout -b feat/korean
$ git add hello.md
$ git commit -m "feat: 인사를 한국어로"
[feat/korean d3acab4] feat: 인사를 한국어로
$ git checkout main
$ git add hello.md
$ git commit -m "feat: 인사를 짧게"
[main 8007611] feat: 인사를 짧게
```

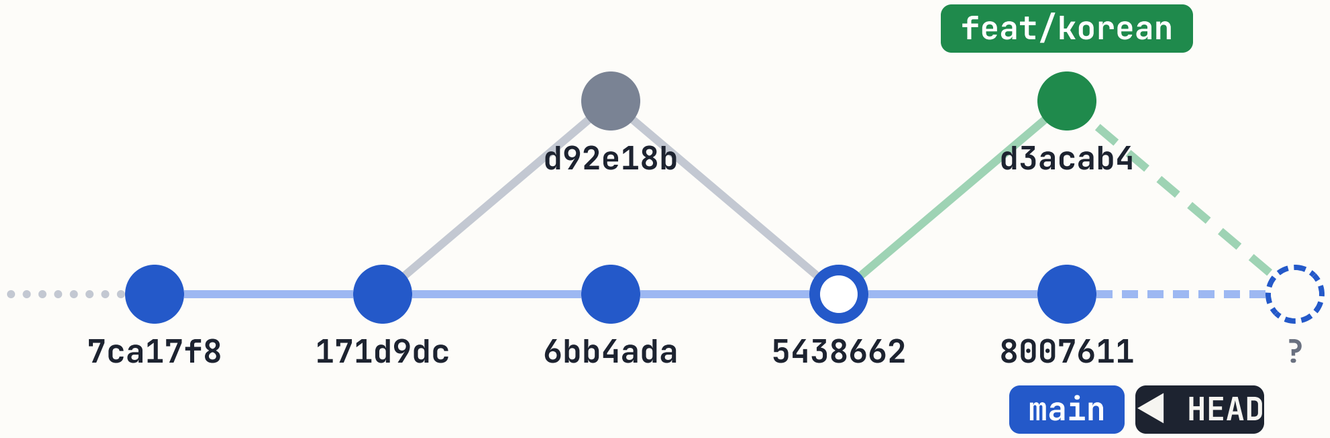
이대로 합치면?

Git은 hi와 안녕 중
무엇을 남길지 몰라요

실습 13 · 충돌

git merge feat/korean

커밋 그래프



작업 폴더

hello.md

```
<<<<<<< HEAD
hi
=======
안녕
>>>>>>> feat/korean
world
nice to meet you
```

```
$ git merge feat/korean
Auto-merging hello.md
CONFLICT (content): Merge conflict in hello.md
Automatic merge failed; fix conflicts and then commit the result.
$ git status
Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified: hello.md
```

합치다 멈췄어요

Git이 파일에 양쪽 내용을
모두 적어 뒀어요

충돌 풀기

충돌 표시 읽는 법

hello.md

```
<<<<<<< HEAD
hi
=======
안녕
>>>>>>> feat/korean
world
nice to meet you
```

지금 내 브랜치(main) 내용

합치려는 브랜치(feat/korean) 내용

푸는 순서

1 남길 내용으로 고치기
둘 다 남겨도 돼요

2 표시 세 줄 지우고 저장

3 `git add hello.md`

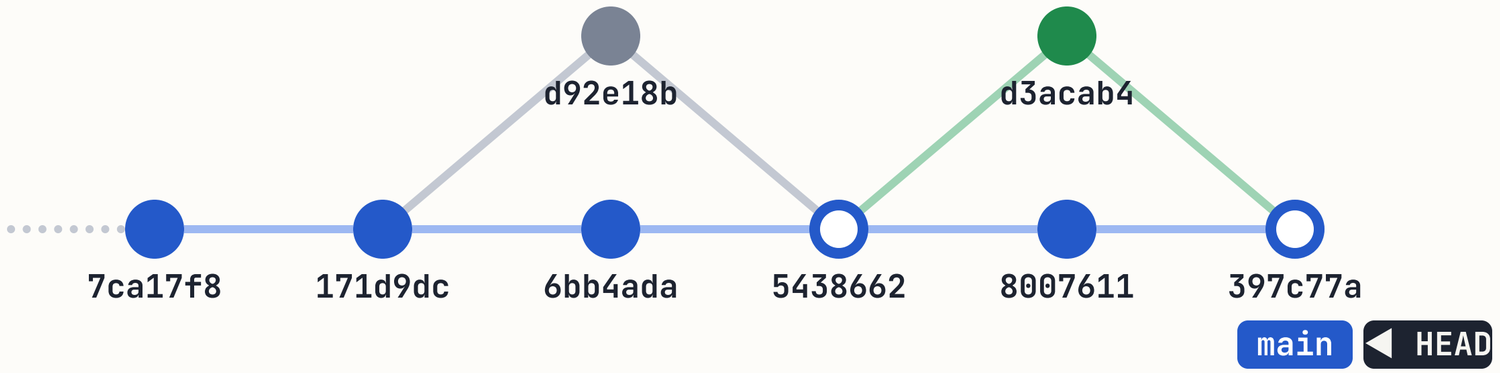
4 `git commit -m "..."`

<<<<<<<, =======, >>>>>>> 줄이 남은 채로 커밋하지 않아요.

실습 14 · 충돌 해결하고 커밋

git commit -m "fix: 인사 충돌 해결"

커밋 그래프



작업 폴더

```
hello.md
안녕
world
nice to meet you
```

```
# 표시 세 줄을 지우고 안녕만 남긴 뒤 저장
$ git add hello.md
$ git status
All conflicts fixed but you are still merging.
$ git commit -m "fix: 인사 충돌 해결"
[main 397c77a] fix: 인사 충돌 해결
$ git branch -d feat/korean
Deleted branch feat/korean (was d3acab4).
```

고친 파일을 add, 그리고 commit

이 커밋이 두 줄기를 이어요

GitHub에서 함께

팀에서는 이 순서로 일해요

내 컴퓨터

1

브랜치 만들기

```
git checkout -b feat/emoji
```

2

작업하고 커밋

```
add, commit
```

3

브랜치 올리기

```
git push origin feat/emoji
```

GitHub 웹사이트

4

합쳐 달라고 요청

```
Pull Request
```

5

확인하고 합치기

```
리뷰 → Merge
```

내 컴퓨터

6

내 main 맞추기

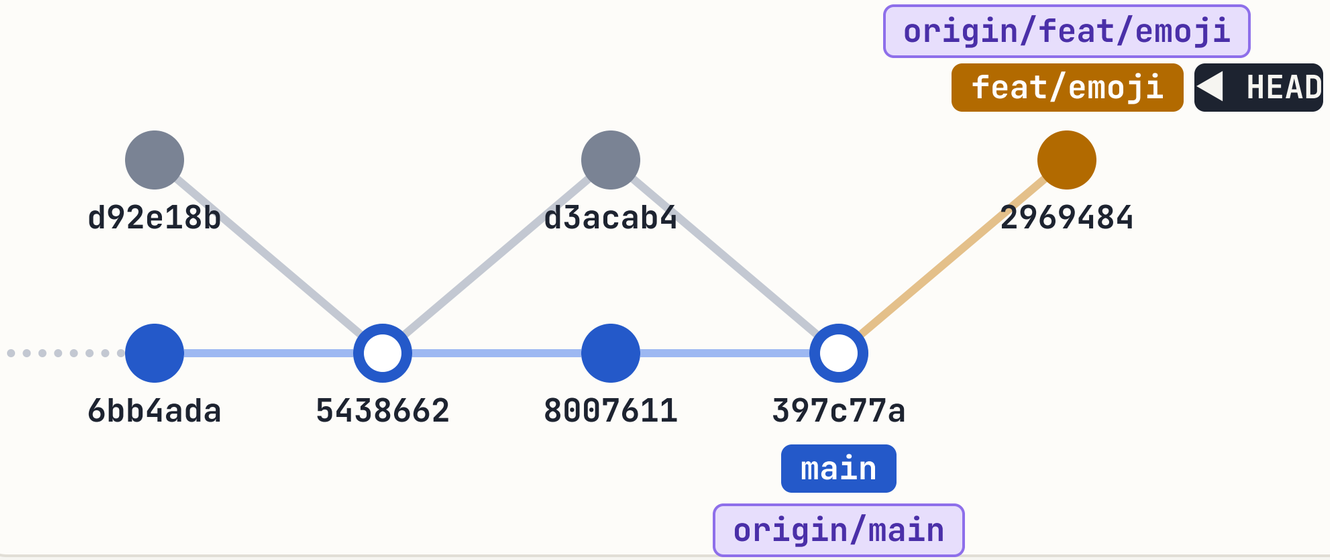
```
git checkout main
git pull origin main
```

main에는 직접 push하지 않고, 합치는 일은 Pull Request로 해요.

실습 15 · 브랜치를 GitHub에 올리기

git push origin feat/emoji

커밋 그래프



작업 폴더

```
bye.md
see you
bye
bye bye
```

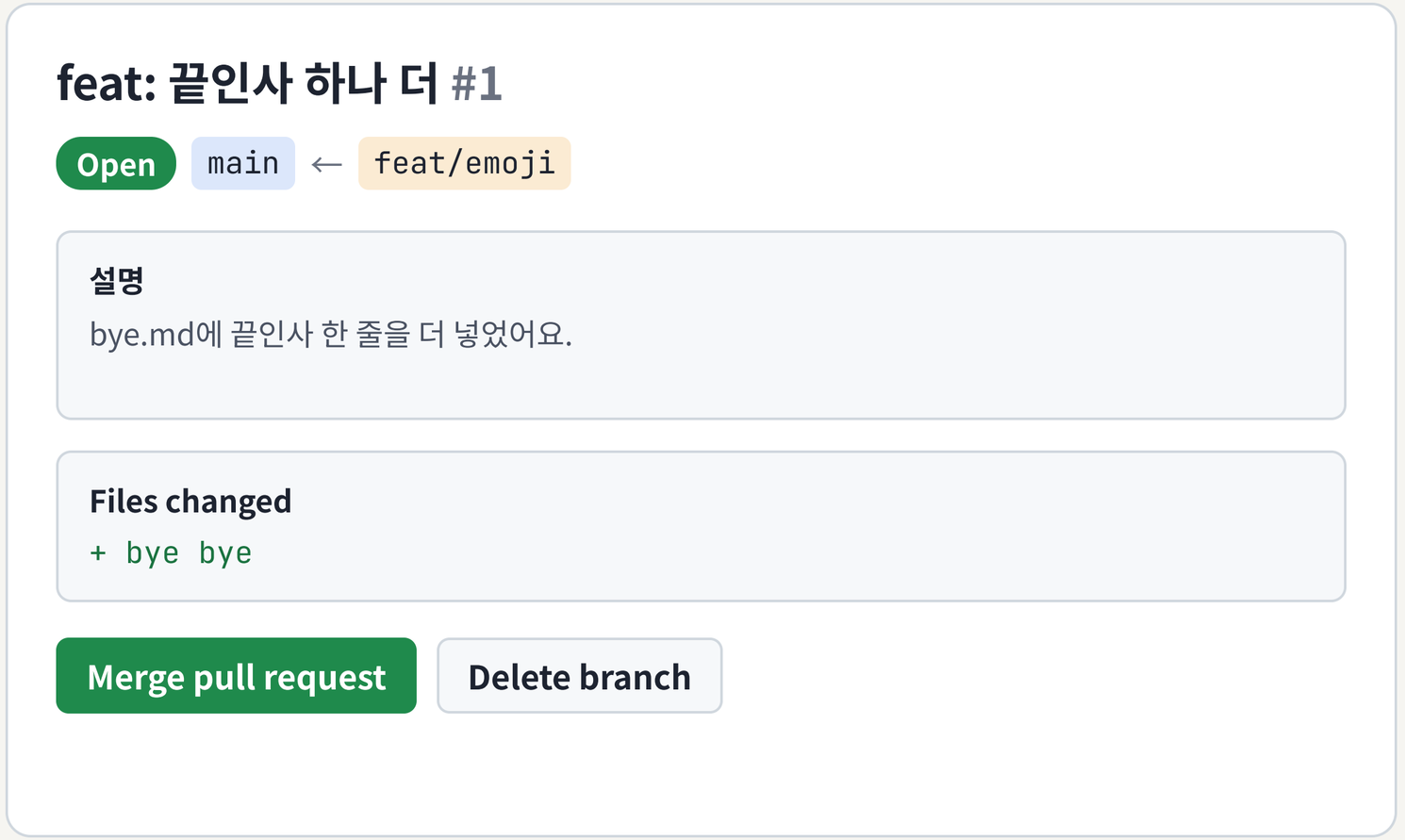
```
$ git checkout -b feat/emoji
$ git commit -am "feat: 끝인사 하나 더"
[feat/emoji 2969484] feat: 끝인사 하나 더
$ git push origin feat/emoji
remote: Create a pull request for 'feat/emoji' on GitHub by visiting:
remote: https://github.com/kim/hello-git/pull/new/feat/emoji
To https://github.com/kim/hello-git.git
 * [new branch] feat/emoji → feat/emoji
```

브랜치째 GitHub으로

-u 없이 origin 브랜치이름까지 끝까지 적어요

GitHub에서 함께

Pull Request 화면에서 할 일

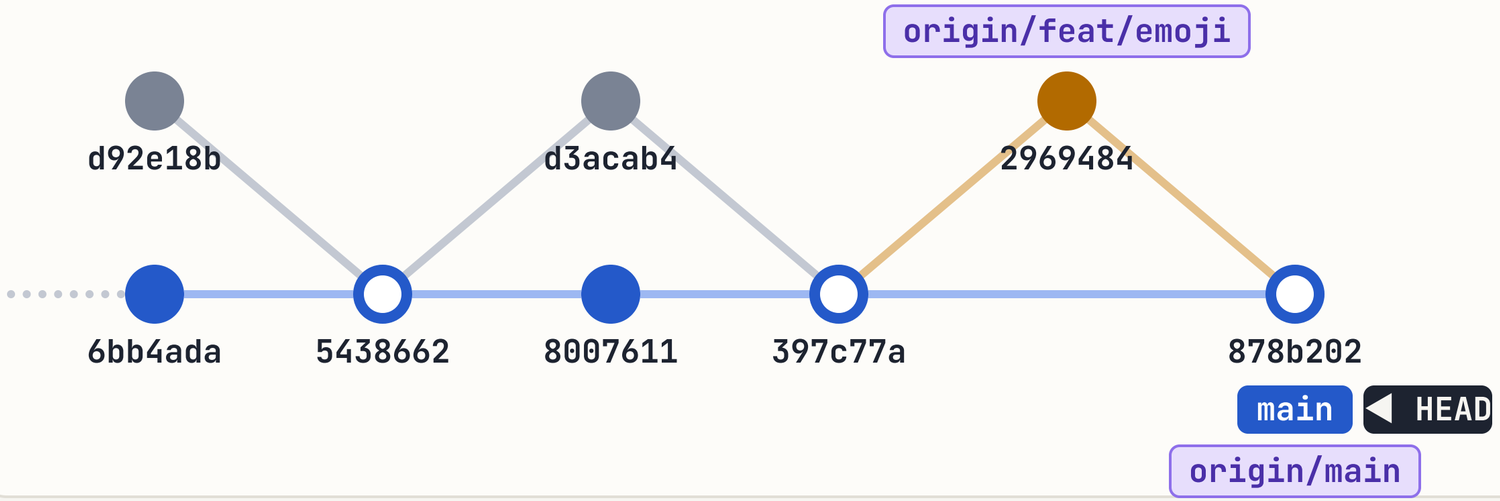


1. 제목: 커밋 메시지처럼 타입: 요약
2. 어디로 ← 무엇을 합치는지
3. 무엇을 왜 바꿨는지
4. 바뀐 줄 확인, 댓글로 리뷰
5. 합치고 나면 GitHub 브랜치 삭제

실습 16 · GitHub에서 merge한 뒤

git checkout main → git pull origin main

커밋 그래프



작업 폴더

```
bye.md
see you
bye
bye bye
```

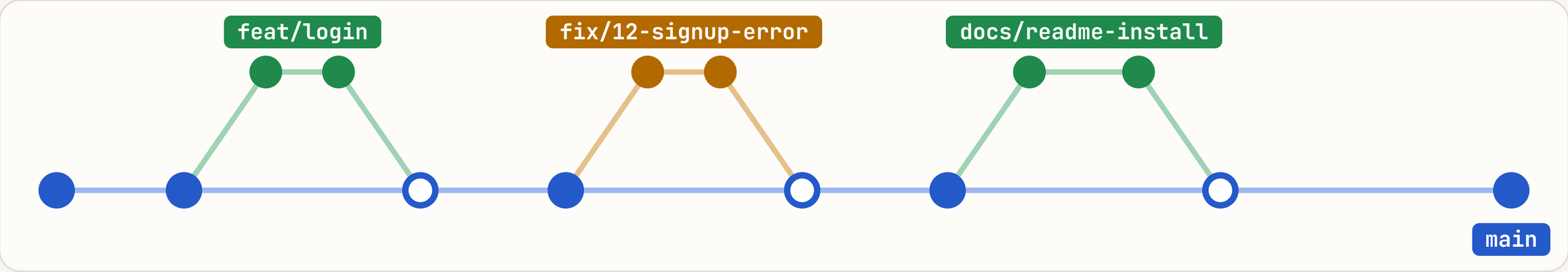
```
$ git checkout main
Switched to branch 'main'
$ git pull origin main
Updating 397c77a..878b202
Fast-forward
 bye.md | 1 +
$ git branch -d feat/emoji
Deleted branch feat/emoji (was 2969484).
```

GitHub의 merge를 내 main으로 받아요

Pull Request merge 커밋이 새로 생겨 있어요

GitHub에서 함께 · 브랜치 전략

우리는 GitHub Flow로 일해요



main은 늘 동작하는 상태.
직접 커밋하지 않아요

일 하나 = 브랜치 하나.
이름은 타입/하는 일

작게 자주 Pull Request

merge 뒤 브랜치는 지워요

Git Flow(develop, release 브랜치를 따로 두는 방식)는 배포 일정이 복잡한 회사에서 써요. 처음엔 GitHub Flow면 충분해요.

에러 1 · 브랜치 이름

옮기려는데, 만들려는데 안 돼요

없는 브랜치로 옮김

```
$ git checkout feat/nope
error: pathspec 'feat/nope' did not match any
file(s) known to git
```

이름 오타이거나 아직 안 만든 브랜치예요

```
git branch
```

```
git checkout -b feat/nope
```

이미 있는 이름으로 만듦

```
$ git checkout -b feat/a
fatal: a branch named 'feat/a' already exists
```

같은 이름이 벌써 있어요. -b 없이 옮겨 가요

```
git checkout feat/a
```

에러 2 · 커밋 안 한 변경

브랜치를 옮기려는데 막혀요

고친 파일이 그대로 있음

```
error: Your local changes to the following files would be overwritten by checkout:
        hello.md
Please commit your changes or stash them before you switch branches.
Aborting
```

옮겨 가면 고친 내용이 덮어써져서 Git이 멈췄어요. 먼저 커밋하고 옮겨요

```
git add hello.md
```

```
git commit -m "feat: ..."
```

```
git checkout feat/b
```

에러 3 · 브랜치 지우기

브랜치가 안 지워져요

아직 합치지 않은 커밋이 있음

```
$ git branch -d feat/a
error: the branch 'feat/a' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feat/a'
```

지우면 그 커밋을 잃을 수 있어서 Git이 멈췄어요

```
git checkout main
```

```
git merge feat/a
```

```
git branch -d feat/a
```

한 장 요약

브랜치

```
git branch
git checkout -b 이름
git checkout 이름
git branch -d 이름
```

합치기

```
git checkout main
git merge 이름
```

충돌

```
파일에서 고르기
표시 세 줄 지우기
git add 파일
git commit -m "..."
```

GitHub과 함께

```
git push origin 이름
→ Pull Request
→ Merge
git checkout main
git pull origin main
```

브랜치 이름: feat/login-button · main에는 직접 커밋하지 않기